

REST API with Node.JS and Express

REST API also known as RESTful API

- **Representational State Transfer** – a s/w development architecture that defines a set of rules for communication between a client and a server.

Lets breakdown a little more

- A REST client is a code or app used to communicate with REST servers
- A server contains resources that the client wants to access or change
- A resource is any information that the API can return

- It conforms to the REST architecture.
- These APIs use HTTP protocol to access and manipulate data on server
- The essential components of a REST API include the **HTTP methods, endpoint, headers and body**

HTTP Methods

- It defines the action the client wants to make on the server, which includes **C**reating, **R**eading, **U**pdating and **D**eleting resources (CRUD) operations.
 - ❑ GET: used to retrieve resources
 - ❑ POST: used to add resources
 - ❑ PUT: used to update resources
 - ❑ DELETE: used to delete resources

Node.js and Express

- Node.js is ideal choice for developers to create fast and powerful applications with RESTful APIs.
- Node.js is an open-source and cross-platform Javascript runtime environment that runs based on chrome's v8 engine.

Why to use Node.js and Express to build REST API?

1. The ability to use a single language (JS) for both client and server- side development.
2. Fast and powerful performance owing to the ability to run multiple requests in parallel.
3. Middleware and Routing capabilities that are built into Express to make API development quick and easy.
4. A large active community of developers contributing to the ecosystem.

Key Concepts of REST API

1. Resources

Resources are the **fundamental units in REST**. Each resource is **identified by a URL** (Uniform Resource Locator).

For example, in a REST API for managing books, a book might be a resource accessible at `/books/1`.

2. HTTP Methods

REST APIs use standard HTTP methods to interact with resources:

- **GET**: Retrieve data from the server (e.g., get a list of books or a single book).
- **POST**: Send data to the server to create a new resource (e.g., add a new book).
- **PUT**: Update an existing resource with new data (e.g., update details of an existing book).
- **DELETE**: Remove a resource from the server (e.g., delete a book).

Key Concepts of REST API

3. Statelessness

Each request from a client to a server must contain all the information the server needs to fulfill that request. The server does not store any state about the client session between requests. This makes REST APIs **scalable and easy to maintain**.

4. Representations

Resources can be represented in **different formats**, such as JSON (JavaScript Object Notation), XML (eXtensible Markup Language), or plain text. **JSON is the most commonly used format** due to its lightweight nature and ease of use.

Key Concepts of REST API

5. Uniform Interface

REST APIs are designed to have a uniform interface that allows for consistent interaction with resources. This involves:

- **Resource identification:** URLs uniquely identify resources.
- **Manipulation of resources through representations:** Clients use representations to interact with resources (e.g., sending JSON data to create or update a resource).
- **Self-descriptive messages:** Each message includes enough information to describe how to process it.
- **Hypermedia as the engine of application state (HATEOAS):** Clients interact with resources entirely through hyperlinks provided dynamically by the server.

Key Concepts of REST API

6. Client-Server Architecture

REST *separates the client and server*, allowing them to *evolve independently*. The *client handles the user interface and user experience, while the server manages data storage, processing, and security*.

7. Cacheability

Responses from a REST API can be explicitly marked as cacheable or non-cacheable. This allows clients to cache responses to improve performance and reduce the need for repeated requests.

An example of Rest API endpoints for a library:

1. **Get a list of all books**

Endpoint: [/books](#)

HTTP Method: GET

Description: Retrieves a list of all books in the library.

Request:

GET /books

Response:

```
[  
  {  
    "id": 1,  
    "title": "1984",  
    "author": "George Orwell",  
    "published_year": 1949  
  },  
  {  
    "id": 2,  
    "title": "To Kill a Mockingbird",  
    "author": "Harper Lee",  
    "published_year": 1960  
  }  
]
```

An example of Rest API endpoints for a library:

2. Get details of a specific book

Endpoint: `/books/{id}`

HTTP Method: GET

Description: Retrieves details of a specific book by its ID.

Request:

GET /books/1

Response:

{

 "author": "George Orwell",

 "published_year": 1949

 "published_year": 1949

}

An example of Rest API endpoints for a library:

3. Add a new book

Endpoint: /books

HTTP Method: POST

Description: Adds a new book to the library.

Request:

POST /books

Content-Type: application/json

{

"title": "Brave New World",

"author": "Aldous Huxley",

"published_year": 1932

}

Response:

{

"id": 3,

"title": "Brave New World",

"author": "Aldous Huxley",

"published_year": 1932

}

An example of Rest API endpoints for a library:

4. Update an existing book

Endpoint: /books/{id}

HTTP Method: PUT

Description: Updates the details of an existing book.

Request:

PUT /books/1

Content-Type: application/json

{

"title": "1984",

"author": "George Orwell",

"published_year": 1948

}

Response:

{

"id": 1,

"title": "1984",

"author": "George Orwell",

"published_year": 1948

}

REST API for a book library

```
// rest.js
const express = require('express');
const app = express();
const PORT = 3000;

// Middleware to parse JSON
// request bodies
app.use(express.json());

// To define the sample data
let books = [
  {
    id: 1,
    title: 'Harry Potter',
    author: 'J. K. Rowling'
  },
  {
    id: 2,
    title: 'Jungle Book',
    author: 'Rudyard Kipling'
  },
];
```

// Define routes for handling GET requests

```
app.get('/api/books', function(req, res) {  
  res.json(books);  
});
```

```
app.get('/api/books/:id', (req, res) => {  
  const id = parseInt(req.params.id);  
  const book = books.find(book => book.id === id);  
  if (book) {  
    res.json(book);  
  }  
  else {  
    res.status(404)  
      .json({ message: 'Book not found' });  
  }  
});
```

This is an **arrow function** that serves as a callback for handling HTTP GET requests to `/api/books/:id`. Here's what it does:

• `(req, res) => { ... }` is a **callback function** that receives two arguments:

- `req` (short for request): Represents the incoming HTTP request and contains request data (e.g., URL parameters, headers, body).

- `res` (short for response): Represents the HTTP response that will be sent back to the client.

• Inside the function:

1. `const id = parseInt(req.params.id);`

- Extracts the `id` parameter from the URL (`:id`) and converts it to an integer.

2. `const book = books.find((book) => book.id === id);`

- Searches for a book in the `books` array where `book.id` matches the extracted `id`.

3. If a book is found:

- `res.json(book);` sends the book data as a JSON response.

4. If no book is found:

- `res.status(404).json({ message: 'Book not found' });` returns a 404 error response.


```
app.listen(PORT,  
  () =>{  
    console.log(`Server is running on port ${PORT}`);  
  });
```

Explanation of New Routes:

// POST - Add a new book

```
app.post('/api/books', (req, res) => {  
  const { title, author } = req.body;  
  if (!title || !author) {  
    return res.status(400).json({ message: 'Title and author are  
required' });  
  }  
  
  const newBook = {  
    id: books.length ? books[books.length - 1].id + 1 : 1, // Auto-  
increment ID  
    title,  
    author  
  };  
  
  books.push(newBook);  
  res.status(201).json(newBook);  
});
```

POST /api/books

- Adds a new book to the collection.
- Expects a request body with title and author.
- Generates a new ID automatically.
- Returns the newly created book.

Explanation of New Routes:

// PUT - Update an existing book by ID

```
app.put('/api/books/:id', (req, res) => {  
  const id = parseInt(req.params.id);  
  const { title, author } = req.body;  
  const bookIndex = books.findIndex(book => book.id === id);  
  
  if (bookIndex === -1) {  
    return res.status(404).json({ message: 'Book not found' });  
  }  
}
```

// Update book properties if provided

```
books[bookIndex] = {  
  ...books[bookIndex],  
  title: title || books[bookIndex].title,  
  author: author || books[bookIndex].author  
};
```

```
res.json(books[bookIndex]);
```

```
});
```

PUT /api/books/:id

- Updates an existing book by ID.
- If title or author is not provided, it keeps the existing values.
- Returns the updated book.

Explanation of New Routes:

// DELETE - Remove a book by ID

```
app.delete('/api/books/:id', (req, res) => {  
  const id = parseInt(req.params.id);  
  const bookIndex = books.findIndex(book => book.id === id);  
  
  if (bookIndex === -1) {  
    return res.status(404).json({ message: 'Book not found' });  
  }  
  
  const deletedBook = books.splice(bookIndex, 1);  
  res.json({ message: 'Book deleted', book: deletedBook[0] });  
});
```

DELETE /api/books/:id

- Deletes a book by ID.
- Returns the deleted book details.

Checking the API via Browser

- The **browser** only supports **GET** requests, so you can test the following endpoints:

GET all books

- Open your browser and visit:

<http://localhost:3000/api/books>

This should return all books in JSON format.

GET a specific book by ID

- Open your browser and visit:

<http://localhost:3000/api/books/1>

If the book with ID 1 exists, you will see its details in JSON format.

If not, you'll get a 404 Book not found message.

Checking the API via Browser

- The browser cannot be used to test POST, PUT, or DELETE requests because it only supports GET requests.

Testing the API:

- You can use **Postman** or **cURL** to test these endpoints:
- Download Postman from <https://www.postman.com/download>
[s/](#)
- A GUI-based tool, **Postman** is a great option for testing all HTTP methods (GET, POST, PUT, DELETE).
- Download – SetUp- Sign up with google
- Click on '+' near overview

Nodemon

- nodemon is a tool for Node.js that automatically restarts your server when you make changes to the code.
- This saves you from manually stopping and restarting the server every time you edit your files.

Why Use nodemon?

- Auto-restart on changes → No need to stop & restart manually.
- Faster development workflow → Instant updates.
- Works with any Node.js project → Especially useful for Express.js applications.

Installing nodemon

- *npm install -g nodemon*

Verify installation:

nodemon -v

To run

Instead of: `node rest.js`

Use: *nodemon rest.js*

- Open **Postman**.
- Enter the request URL (e.g., `http://localhost:3000/api/books`).
- Select the request method (GET, POST, PUT, DELETE).
- For **POST** and **PUT**, go to the **Body** tab, select **raw**, and choose **JSON** format.
- Enter the request body (e.g., `{"title": "New Book", "author": "Author Name"}`).
- Click **Send** to get a response.

Checking the API via Terminal (cURL)

You can use the **terminal (Command Prompt, Bash, or PowerShell)** with the cURL command to test all endpoints.

✓ GET all books

sh

```
curl -X GET http://localhost:3000/api/books
```

✓ GET a book by ID

sh

```
curl -X GET http://localhost:3000/api/books/1
```

✓ POST (Add a new book)

sh

```
curl -X POST http://localhost:3000/api/books \ -H "Content-Type: application/json" \ -d '{"title":"1984", "author":"George Orwell"}'
```

- This adds a new book with the given title and author.

✓ PUT (Update a book)

sh

```
curl -X PUT http://localhost:3000/api/books/1 \ -H "Content-Type: application/json" \ -d '{"title":"The Great Gatsby (Updated)"}'
```

- This updates the title of the book with ID 1.

✓ DELETE (Remove a book)

sh

```
curl -X DELETE http://localhost:3000/api/books/1
```

- This deletes the book with ID 1.